

Retrieval-Augmented Generation (RAG)

From Retrieval to Generation

Lewis et al. (2020) | Facebook AI Research | [arXiv:2005.11401](#)

What is RAG?

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020) at Facebook AI Research, combines parametric memory (the LLM's weights) with non-parametric memory (a retrieved document store) to produce more accurate, up-to-date, and verifiable responses.

1. Motivation & Problem Statement

Standard language models store knowledge implicitly in parameters, making knowledge updates expensive (requiring retraining) and hallucinations hard to eliminate. RAG decouples knowledge storage from the model, enabling real-time retrieval of relevant passages.

2. RAG Architecture

The RAG pipeline has three components:

Retriever: A dense retrieval model (typically DPR — Dense Passage Retrieval) that encodes the query and document corpus into a shared embedding space. At inference time, the top-k most similar documents are fetched using MIPS (Maximum Inner Product Search).

Generator: A sequence-to-sequence model (originally BART) that conditions on the concatenation of the query and retrieved documents to produce the final answer.

Document Index: A pre-built FAISS index over encoded document chunks, enabling sub-linear time approximate nearest neighbor search over millions of passages.

3. RAG Variants

Variant	Description	Use Case
RAG-Sequence	Retrieve once, generate full answer	Open-domain QA
RAG-Token	Retrieve per token generation step	Knowledge-intensive tasks
Naive RAG	Simple retrieval + prompt stuffing	Quick prototypes
Advanced RAG	Pre/post-retrieval optimization	Production systems

Modular RAG	Plug-in retrieval modules	Flexible pipelines
Graph RAG (MS)	Knowledge graph-guided retrieval	Complex reasoning

4. Implementation with LangChain / LlamaIndex

Basic RAG pipeline using LangChain:

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import Anthropic

# 1. Embed and index documents
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(documents, embeddings)

# 2. Create retriever
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

# 3. Build RAG chain
qa_chain = RetrievalQA.from_chain_type(
    llm=Anthropic(model="claude-3-5-sonnet"),
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)

# 4. Query
result = qa_chain({"query": "What is constitutional AI?"})
print(result["result"])
```

5. Chunking Strategies

- Fixed-size chunking:** Split text every N tokens (e.g., 512) with overlap (e.g., 50 tokens)
- Semantic chunking:** Split on paragraph/sentence boundaries; preserve semantic units
- Recursive character splitting:** LangChain's default; tries multiple separators hierarchically
- Document-structure splitting:** Use markdown headers, HTML tags, or PDF section boundaries
- Small-to-big retrieval:** Index small chunks; retrieve parent document on match

6. Evaluation Metrics

Metric	Measures	Tool
Faithfulness	Are claims grounded in retrieved docs?	RAGAS, TruLens

Answer Relevancy	Does answer address the question?	RAGAS
Context Precision	Are retrieved chunks relevant?	RAGAS
Context Recall	Are all relevant passages retrieved?	RAGAS
MRR / NDCG	Ranking quality of retriever	Custom eval

Reference: Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401